Test report 8 – CLI for agents

cad-cli, cad-render, cad-session · 2026-10-23 · Branch worktree-agent-a1d075a147b8bbdd0 · Commit 8c9876a · rustc 1.97.0 (2d8144b78 2026-07-07) · 4 cores (Intel(R) Xeon(R) Processor @ 2.10GHz)

cargo test --workspace	new tests	cargo clippy -D warnings	cargo fmt --check	render per call	watch run
318 / 318 0 ignored	**20** agent.rs, features.rs, geom	**clean**	**clean**	**1.10 s** max. (2 runs × 3 files)	**30 ms** comment, everything from the cache

Scope

The CLI `cad` is extended so that an agent (e.g. Claude Code) can develop and check models entirely without the UI – alone or while a human edits the same files in the UI. New:

Command/option	Purpose
`cad outline`	Overview of a project in one call
`render --views`	Contact sheet with labelled tiles at the same scale
`render --azimuth/--elevation/--zoom/--target/--fit/--perspective`	Free camera
`render --part/--line (repeatable)/--node/--only`	Selection, combinable with `--module`, `-D`
`render --show grids,anchors,handles,edges,labels,bbox,dims`	Overlays
`render --section/--section-plane/--section-flip`, `--xray`	Section with filled cap, X-ray
`cad measure`	Distances (anchors, grid points, coordinates), bounding box, volume, area, collisions, part clearance
`cad watch`	Re-evaluate on file change, one JSON line per run, optional image
`cad set`	Change a default value or literal through the engine, checked, atomic, `--dry-run`
`cad DATEI.scad`, `cad`	UI entry point reserved (exit 2 with message), help without arguments

Every command has `--json` with a fixed, versioned schema (`schema_version` 1, complete in the header of `crates/cad-cli/src/main.rs`, short form in the skill `.claude/skills/cad-cli/SKILL.md`) and the existing exit codes (0 ok, 1 model error, 2 invocation, 3 kernel). Recommended workflow for agents: **outline → edit → check → render --views all --show grids,anchors → inspect/measure → export**. `cad-render` draws labels with an embedded bitmap font (DejaVu Sans Mono, generated by `tools/font/gen-font.py`), without a new runtime dependency.

1 cad outline

One call returns the structure of a project: modules with file:line, parameters (default value, inferred type), grids and anchors per module (evaluated with the default values, expression planes with expression and value), functions, top-level variables with values, named instances and parts (bounding box, volume), `include`/`use` and the diagnostic count. `--no-kernel` omits bounding boxes and volumes.

```
$ cad outline examples/projekt-gehaeuse/gehaeuse.scad                    275 ms, Exit 0
```

```
examples/projekt-gehaeuse/gehaeuse.scad: 0 Fehler, 0 Warnungen
Dateien:
```

```
  include <konstanten.scad>  gehaeuse.scad:3
  use <lib/teile.scad>  gehaeuse.scad:4
Module:
  schraubdom(h = 10: Zahl)  lib/teile.scad:6–12
    Anker: oben = [0, 0, 10] dir [0, 0, 1] (Z.11)
  gehaeuse(w = 60: Zahl, d = 40: Zahl, h = 25: Zahl, t = 2: Zahl)  lib/teile.scad:14–30
    Raster g (Z.15): x: links = 0, innen_l = 2, innen_r = w - t → 58, rechts = 60; y: vorne = 0, innen_v = 2,
innen_h = d - t → 38, hinten = 40; z: boden = 0, innen_b = 2, oben = 25
    Anker: deckel_mitte = [2, 2, 25] dir [0, 0, 1] (Z.29)
  deckel(w = 60: Zahl, d = 40: Zahl, t = 2: Zahl)  lib/teile.scad:32–35
Variablen:
  breite = 60  konstanten.scad:2
  tiefe = 40  konstanten.scad:3
  hoehe = 25  konstanten.scad:4
  wand = 2  konstanten.scad:5
  abstand = 10  konstanten.scad:6
Benannte Instanzen:
  gehaeuse: gehaeuse  Z.6  Bauteil, Hülle [0, 0, 0]–[60, 40, 25], Volumen 15474.7326 mm³
  dom[0]: schraubdom  Z.27
  dom[1]: schraubdom  Z.27
  dom[2]: schraubdom  Z.27
  dom[3]: schraubdom  Z.27
  deckel: deckel  Z.7  Bauteil, Hülle [0, 50, 0]–[60, 90, 5], Volumen 10738.08 mm³
Anker: gehaeuse.deckel_mitte = [2, 2, 25] dir [0, 0, 1], dom[0].oben = [5, 5, 21] dir [0, 0, 1], dom[1].oben =
[55, 5, 21] dir [0, 0, 1], dom[2].oben = [5, 35, 21] dir [0, 0, 1], dom[3].oben = [55, 35, 21] dir [0, 0, 1]
Gesamt: Hülle [0, 0, 0]–[60, 90, 25] (60 × 90 × 25 mm), 33 Knoten
```

$ cad outline crates/cad-cli/tests/data/gehaeuse.scad --json 45 ms, Exit 0

JSON (EXCERPT: MODULES, PARTS, COUNTS)


```
{
  "modules": [
    {
      "anchors": [
        {"dir": null, "line": 15, "name": "ecke", "point": [40, 30, 20]},
        {"dir": [0, 0, 1], "line": 16, "name": "oben", "point": [0, 0, 20]}
      ],
      "end_line": 17,
      "errors_with_defaults": 0,
      "file": "gehaeuse.scad",
      "grids": [
        {
          "file": "gehaeuse.scad",
          "line": 4,
          "name": "g",
          "planes": [
            {"axis": "x", "expr": "0", "kind": "literal", "name": "links", "value": 0},
            {"axis": "x", "expr": "t", "kind": "literal", "name": "innen_l", "value": 3},
            {"axis": "x", "expr": "w - t", "kind": "expression", "name": "innen_r", "value": 37},
            "… (5 weitere)"
          ]
        }
      ],
      "line": 3,
      "name": "gehaeuse",
      "params": [
        {"default": "40", "default_value": 40, "name": "w", "type": "number"},
        {"default": "30", "default_value": 30, "name": "d", "type": "number"},
        {"default": "20", "default_value": 20, "name": "h", "type": "number"},
        "… (1 weitere)"
      ],
      "path": [],
      "qualified": "gehaeuse"
    },
    {
      "anchors": [],
      "end_line": 19,
```

```
      "errors_with_defaults": 0,
      "file": "gehaeuse.scad",
      "grids": [],
      "line": 19,
      "name": "deckel",
      "params": [],
      "path": [],
      "qualified": "deckel"
    }
  ],
  "parts": [
    {
      "bounds": {"max": [60, 30, 20], "min": [0, 0, 0], "size": [60, 30, 20]},
      "line": 21,
      "module": "gehaeuse",
      "name": "k",
      "node": 1,
      "volume": 8460
    }
  ],
  "counts": {"errors": 0, "modules": 2, "nodes": 12, "parts": 1, "warnings": 0}
}
```


2 cad render: contact sheet

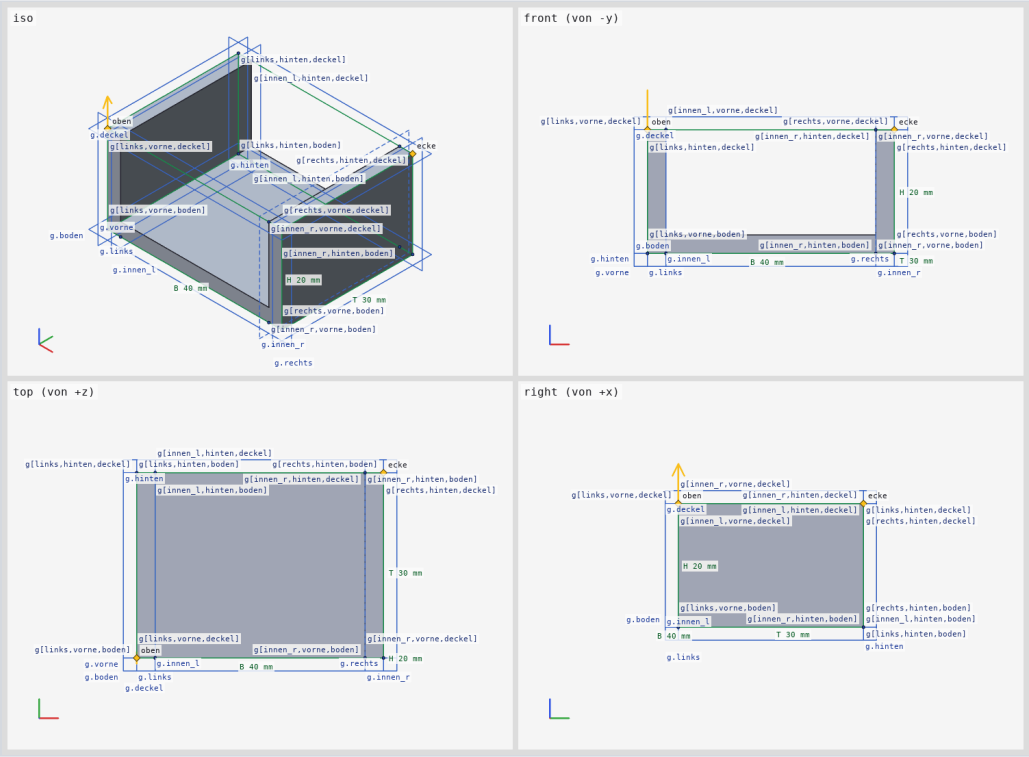
`--views iso,front,top,right` (or `all`) combines labelled tiles into one image. All tiles have the same scale (px/mm) and the same target point (center of the model), so sizes are comparable between views; `--size` is the total size. The JSON gives position and camera per tile.

```
$ cad render crates/cad-cli/tests/data/gehaeuse.scad -o docs/reports/08-cli-agenten/img/bogen.png --module gehaeuse --views all --show "grids,anchors,labels,dims" --size 1500x1100 --json
```
166 ms, Exit 0

JSON (EXCERPT: WIDTH, HEIGHT, VIEWS, OVERLAYS)

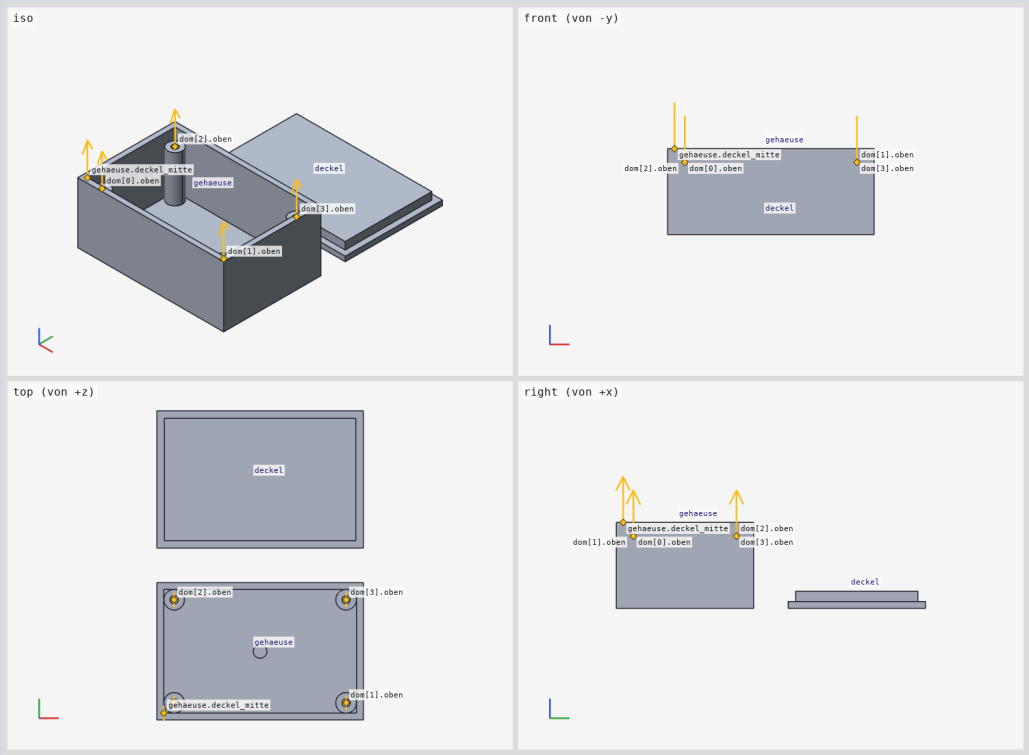

```
{
  "width": 1500,
  "height": 1100,
  "views": [
    {
      "camera": {
        "azimuth": 45,
        "dir": [-0.5774, 0.5774, -0.5774],
        "elevation": 35.2644,
        "eye": [67.3148, -32.3148, 57.3148],
        "fov_deg": null,
        "height_mm": 59.7425,
        "projection": "ortho",
        "right": [0.7071, 0.7071, 0],
        "scale_px_per_mm": 9.0053,
        "target": [20, 15, 10],
        "up": [-0.4082, 0.4082, 0.8165],
        "width_mm": 81.9516
      },
      "height": 538,
      "name": "iso",
      "width": 738,
      "x": 8,
      "y": 8
    },
    {
      "camera": {
        "azimuth": 0,
        "dir": [0, 1, 0],
        "elevation": 0,
        "eye": [20, -66.9516, 10],
        "fov_deg": null,
```

```
        "height_mm": 59.7425,
        "projection": "ortho",
        "right": [1, 0, 0],
        "scale_px_per_mm": 9.0053,
        "target": [20, 15, 10],
        "up": [0, 0, 1],
        "width_mm": 81.9516
      },
      "height": 538,
      "name": "front",
      "width": 738,
      "x": 754,
      "y": 8
    },
    {
      "camera": {
        "azimuth": 0,
        "dir": [0, 0, -1],
        "elevation": 90,
        "eye": [20, 15, 91.9516],
        "fov_deg": null,
        "height_mm": 59.7425,
        "projection": "ortho",
        "right": [1, 0, 0],
        "scale_px_per_mm": 9.0053,
        "target": [20, 15, 10],
        "up": [0, 1, 0],
        "width_mm": 81.9516
… (11 Zeilen mehr)
```

Spec example gehaeuse in module context: grid g with plane names and grid points g[a,b,c], anchors as diamonds with direction arrow, dimensions W/D/H.

```
$ cad render examples/projekt-gehaeuse/gehaeuse.scad -o docs/reports/08-cli-agenten/img/bogen-projekt.png --
views "iso,front,top,right" --show "anchors,labels" --size 1500x1100 --json                    656 ms, Exit 0
```

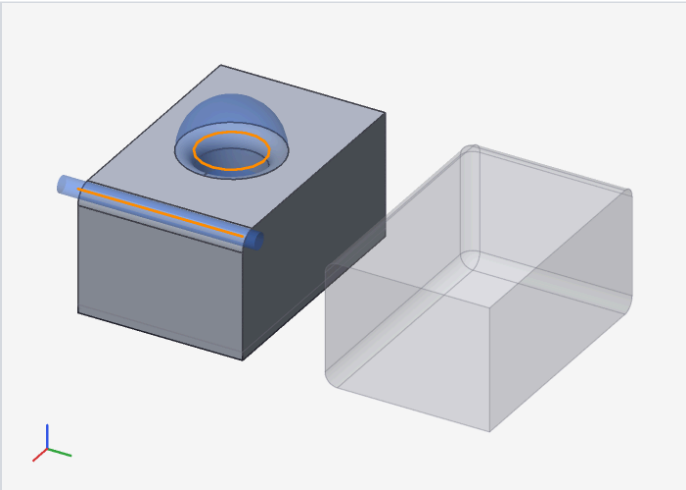


Project of three files in file context: anchors of the named instances (gehaeuse.deckel_mitte, dom[i].oben) and part names.

3 cad render: free camera and selection

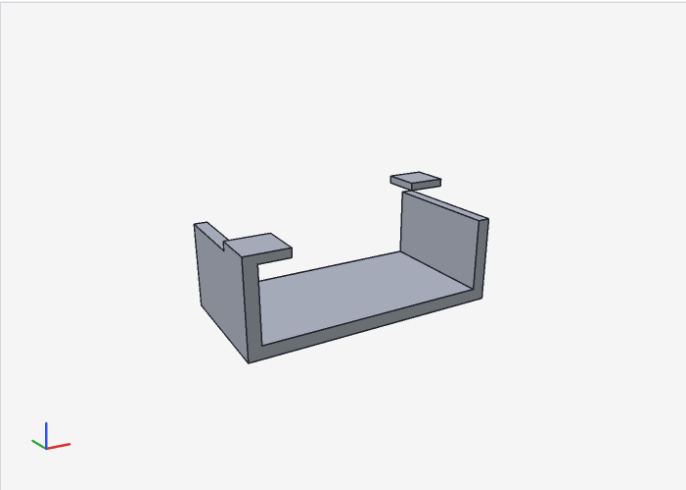
Camera: `--azimuth` (0° from the front/−y, 90° from the right/+x), `--elevation` (90° from above), `--zoom`, `--target x,y,z`, `--fit all|part:NAME|line:N`, `--ortho`/`--perspective [--fov]`. Selection: `--part NAME` (parts only), `--line N` and `--node I` repeatable (nodes in full, the rest dimmed, `--only` omits the rest), combinable with `--module` and `-D`.

```
$ cad render examples/spec/fillet_zonen.scad -o
docs/reports/08-cli-agenten/img/kamera-azimut.png --
line 12 --azimuth 120 --elevation 30 --size 700x500 -
-json                                    525 ms, Exit 0
```



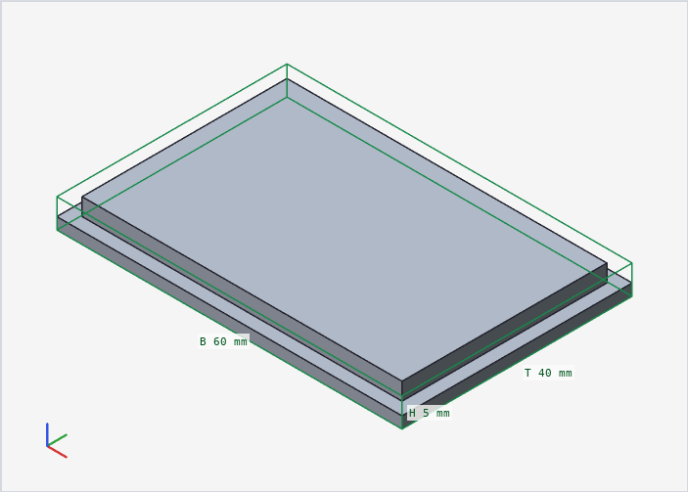
fillet_zonen.scad --line 12 --azimuth 120 --elevation 30: zones blue, selected edges orange.

```
$ cad render crates/cad-cli/tests/data/gehaeuse.scad
-o docs/reports/08-cli-agenten/img/kamera-
perspektive.png --perspective --fov 45 --azimuth -30
--elevation 20 --size 700x500 --json     74 ms, Exit 0
```



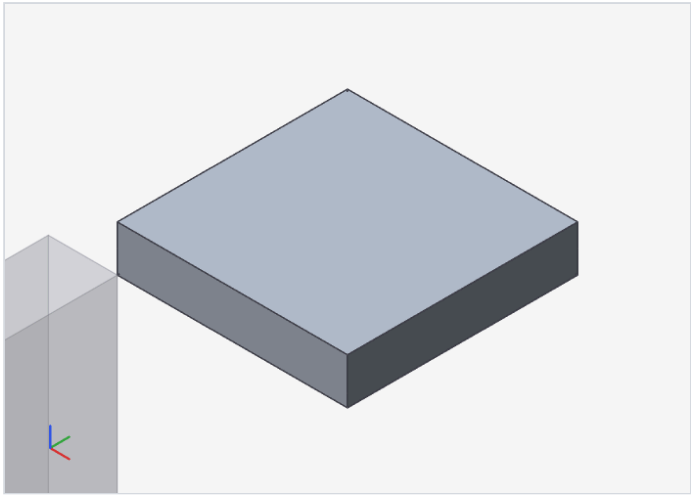
gehaeuse --perspective --fov 45 --azimuth -30 --elevation 20

```
$ cad render examples/projekt-gehaeuse/gehaeuse.scad -o docs/reports/08-cli-agenten/img/auswahl-part.png --part deckel --show dims --size 700x500 --json
```
337 ms, Exit 0



Project --part deckel --show dims

```
$ cad render crates/cad-cli/tests/data/gehaeuse.scad -o docs/reports/08-cli-agenten/img/auswahl-zeilen.png --line 22 --line 23 --fit line:22 --zoom 0.8 --size 700x500 --json
```
159 ms, Exit 0



--line 22 --line 23 --fit line:22: two lines highlighted, fitted to line 22.

```
$ cad render crates/cad-cli/tests/data/gehaeuse.scad -o docs/reports/08-cli-agenten/img/kamera-perspektive.png --perspective --fov 45 --azimuth -30 --elevation 20 --size 700x500 --json
```
74 ms, Exit 0

JSON (EXCERPT: CAMERA)


```
{
  "camera": {
    "azimuth": -30,
    "dir": [0.4698, 0.8138, -0.342],
    "elevation": 20,
    "eye": [-34.1027, -96.0291, 60.1489],
    "fov_deg": 45,
    "height_mm": 116.2131,
    "projection": "perspective",
    "right": [0.866, -0.5, 0],
    "scale_px_per_mm": 4.3024,
    "target": [31.8081, 18.1317, 12.1698],
    "up": [0.171, 0.2962, 0.9397],
    "width_mm": 162.6983
  }
}
```


```
$ cad render crates/cad-cli/tests/data/gehaeuse.scad -o docs/reports/08-cli-agenten/img/auswahl-zeilen.png --line 22 --line 23 --fit line:22 --zoom 0.8 --size 700x500 --json
```
159 ms, Exit 0

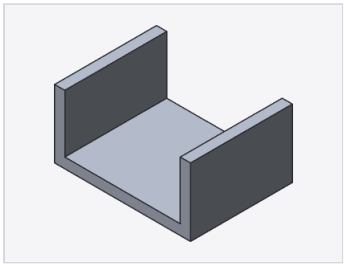
JSON (EXCERPT: HIGHLIGHTED, VISIBLE_NODES)


```
{
  "highlighted": [5, 6, 8, "… (2 weitere)"],
  "visible_nodes": [
    {"file": null, "idx": 0, "instance": null, "label": "group", "line": 1, "role": "dimmed", "triangles": 54},
    {
      "file": null,
      "idx": 5,
      "instance": null,
      "label": "translate",
      "line": 22,
      "role": "highlighted",
      "triangles": 12
    },
    {
      "file": null,
      "idx": 6,
      "instance": null,
```

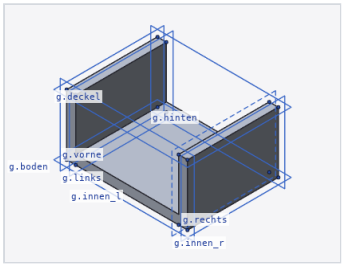
```
      "label": "deckel",
      "line": 22,
      "role": "highlighted",
      "triangles": 12
    },
    "… (3 weitere)"
  ]
}
```


4 cad render: overlays

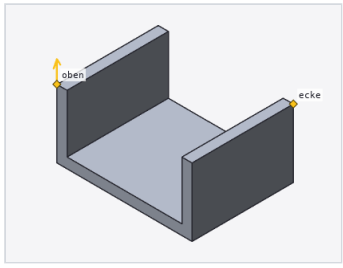
`--show grids,anchors,handles,edges,labels,bbox,dims` (or `all`): grids of the context module (planes as rectangles at the edge of the grid, expression planes dashed, intersections as dots, plane names `g.links`), anchors as diamonds with name and arrow for `dir`, handles as squares, hidden edges dashed; `labels` labels grid points `g[a,b,c]` (up to 48 points per grid), handles and parts, `bbox` draws the bounding box, `dims` additionally width/depth/height in mm. Labels that would overlap are shifted or omitted.



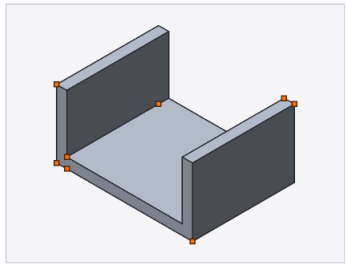
no overlay



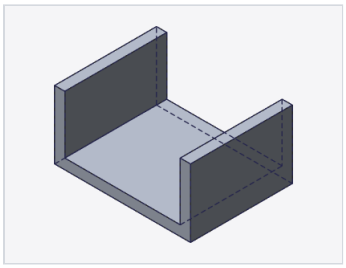
--show grids



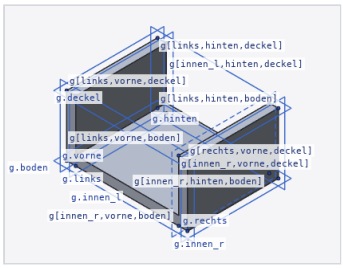
--show anchors



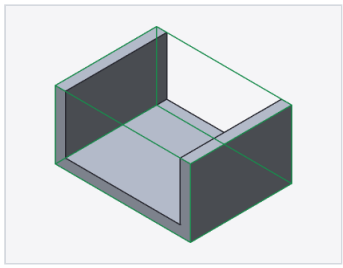
--show handles



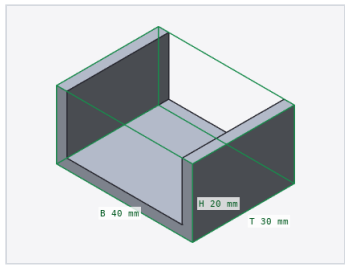
--show edges



--show grids,labels



--show bbox



--show dims

--show	Grids	Grid points	Anchors	Handles	Labels	Lines
grids	1	16	0	0	8	8
anchors	0	0	2	0	2	2
handles	0	0	0	13	0	0
edges	0	0	0	0	0	24
grids,labels	1	16	0	0	24	8
bbox	0	0	0	0	0	12
dims	0	0	0	0	3	12

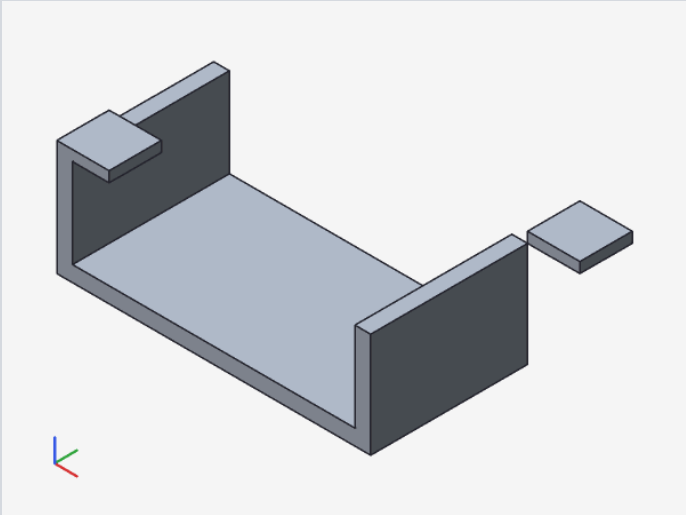
5 cad render: section and X-ray

Section: `--section x=WERT|y=WERT|z=WERT`, `--section-plane px,py,pz,nx,ny,nz`, `--section-flip`. Convention (the same for the later UI): plane through *p* with normal *n*; the half that *n* points into is removed, (q − p)·n ≤ 0 remains. `z=10` means p = (0,0,10), n = +z; `--section-flip` reverses n. The section face is drawn as a hatched cap in its own color (hatching alternates per body), the section contour as a line.

Method: clipping in the renderer with a cap, no kernel boolean. The triangles are clipped geometrically at the plane (Sutherland–Hodgman per triangle, edges likewise). The cap follows the GPU stencil principle: for each

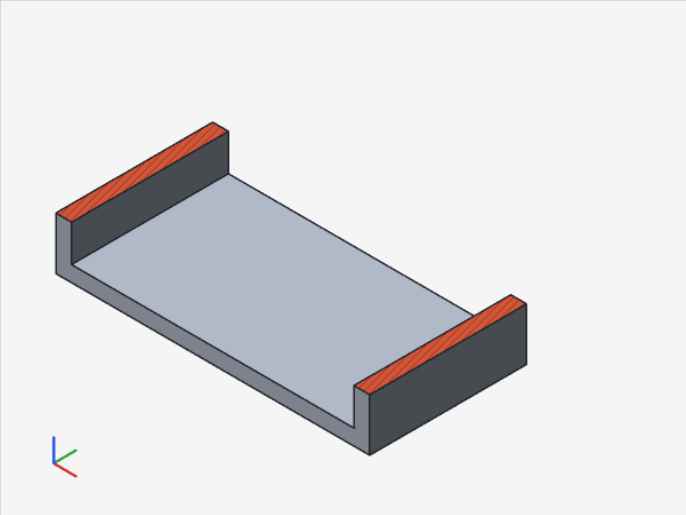
sample, the number of times the view ray crosses the surface of a body up to the plane is counted (top-left rule, so shared edges count exactly once); if the count is odd, the plane point lies inside the body and the cap is written at plane depth. Reasons: (1) the UI will use the same method with GPU clipping and a stencil cap, so images and convention match; (2) it needs no kernel work – a boolean with a half-space quickly costs seconds for filleted bodies and can fail, clipping costs a few ms and always succeeds; (3) it works for any plane, also for views of several parts. It requires closed meshes; the tessellation of the B-rep bodies is closed. Open surfaces (`surface`) and transparent bodies get no cap.

```
$ cad render crates/cad-cli/tests/data/gehaeuse.scad -o docs/reports/08-cli-agenten/img/schnitt-ohne.png --size 600x450 --json
```
82 ms, Exit 0



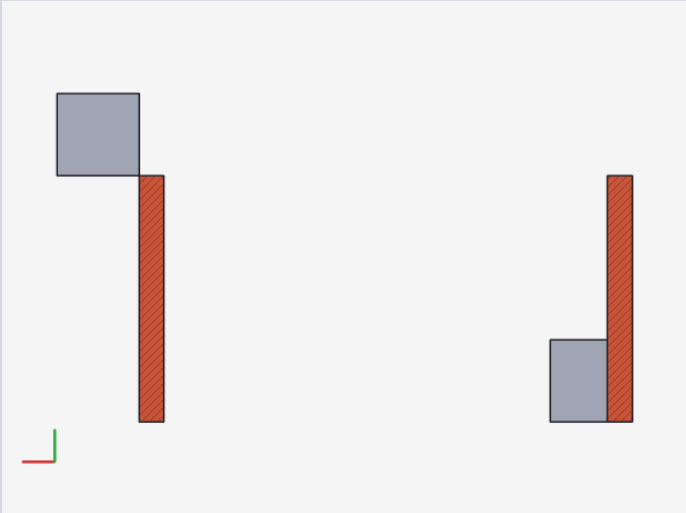
no section

```
$ cad render crates/cad-cli/tests/data/gehaeuse.scad -o docs/reports/08-cli-agenten/img/schnitt-z.png --section z=10 --size 600x450 --json
```
121 ms, Exit 0



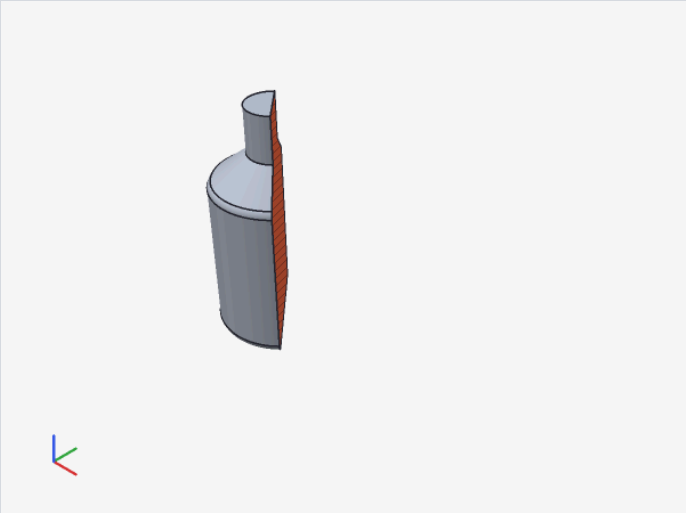
--section z=10: upper half removed, walls cut

```
$ cad render crates/cad-cli/tests/data/gehaeuse.scad -o docs/reports/08-cli-agenten/img/schnitt-flip.png --section z=10 --section-flip --view bottom --size 600x450 --json
```
86 ms, Exit 0



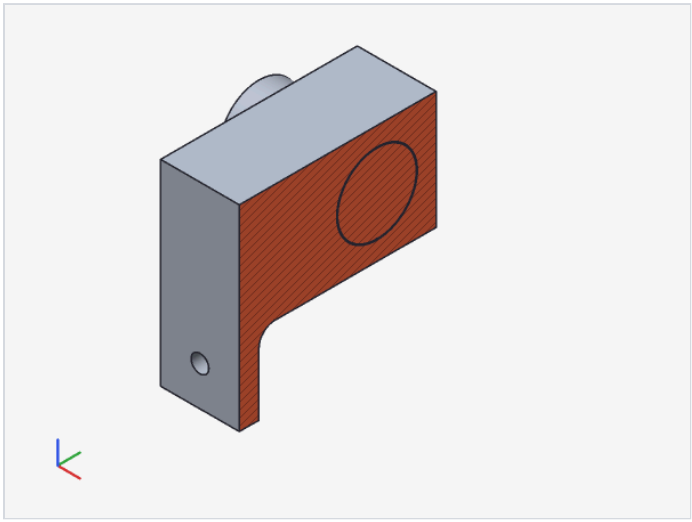
--section z=10 --section-flip --view bottom: upper half remains

```
$ cad render examples/spec/flasche.scad -o docs/reports/08-cli-agenten/img/schnitt-ebene.png --section-plane "0,0,0,1,1,0" --perspective --size 600x450 --json
```
444 ms, Exit 0



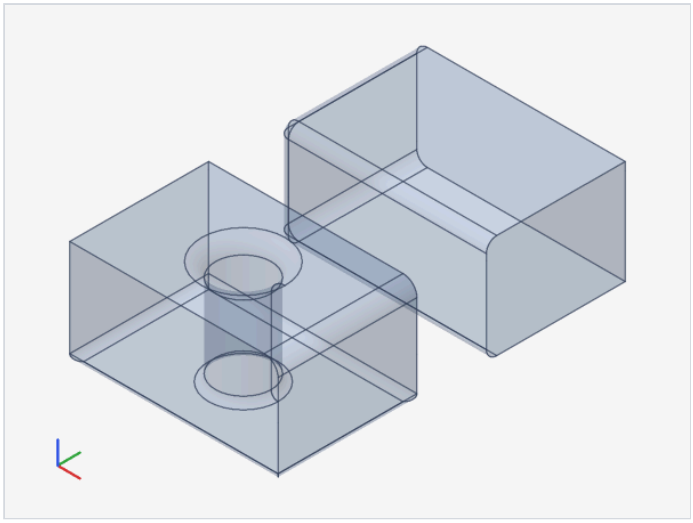
flasche.scad --section-plane 0,0,0,1,1,0 --perspective

```
$ cad render tools/report/report8-fallstudie/v2.scad -o docs/reports/08-cli-agenten/img/schnitt-fallstudie.png --section x=20 --size 600x450 --json
                                                                                                                                                  395 ms, Exit 0
```



Case study --section x=20: rod in the bore, two caps with opposite hatching, 0.25 mm gap as contour

```
$ cad render examples/spec/fillet_zonen.scad -o docs/reports/08-cli-agenten/img/xray.png --xray --size 600x450 --json
                                                                                                                                                  389 ms, Exit 0
```



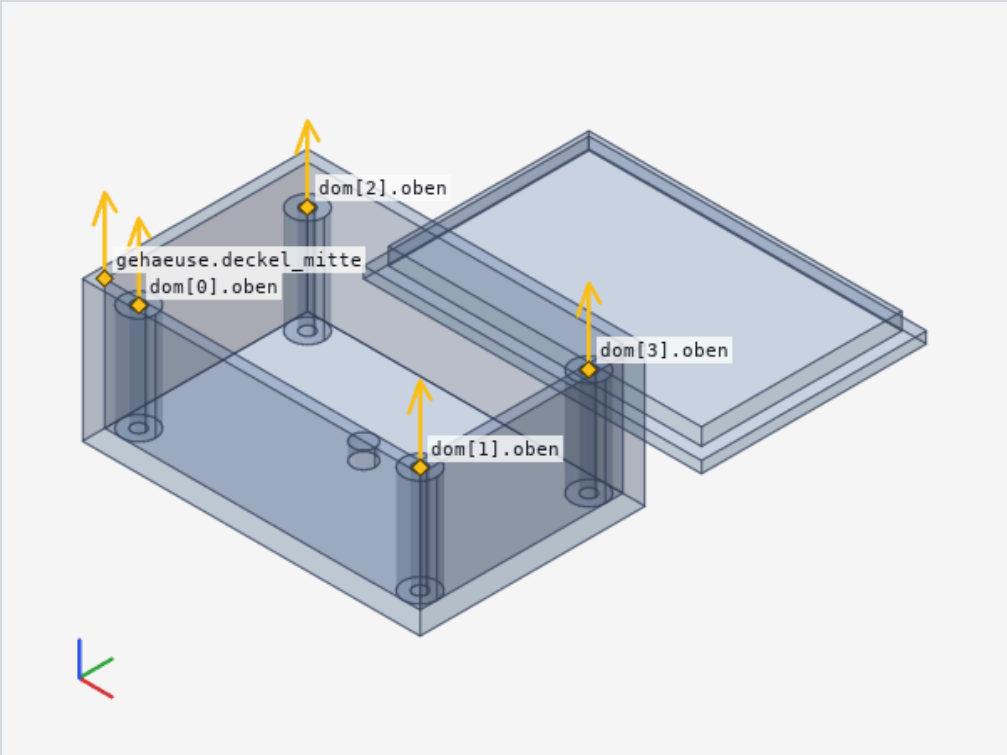
fillet_zonen.scad --xray: everything semi-transparent with edges

```
$ cad render crates/cad-cli/tests/data/gehaeuse.scad -o docs/reports/08-cli-agenten/img/schnitt-z.png --section z=10 --size 600x450 --json
                                                                                                                                                  121 ms, Exit 0
```

JSON (EXCERPT: SECTION, BOUNDS)


```
{
  "section": {
    "keeps": "(q - point) · normal <= 0",
    "method": "renderer clipping + cap",
    "normal": [0, 0, 1],
    "point": [0, 0, 10]
  },
  "bounds": {"max": [70, 40, 22], "min": [0, 0, 0], "size": [70, 40, 22]}
}
```


Visible area (pixels ≠ background, 600×450): without section 74,159, with `--section z=10` 58,204 (78 %), of which cap 3,967. The same comparison is the test `render_section_reduces_visible_area`.



Project --xray --show anchors: screw bosses and anchors visible through the wall

6 cad measure

`--between A B` with anchors (`k.ecke`, `k[2].ecke`, in module context `ecke`), grid points of the context module (`g[a,b,c]`, entries may also be numbers or _) and coordinates `[x,y,z]`: distance and difference vector. `--bounds/--volume/--area ZIEL` with `all`, `part:NAME`, `line:N`, `node:I`. `--interference`: pairwise intersection volume of all parts, exact via the kernel (boolean intersection of the B-rep, bounding boxes tested first); collision from `--tolerance` (default 0.001 mm³). `--clearance A B`: smallest distance between two parts as the exact triangle–triangle distance of their tessellations (BVH, tolerance `--mesh-tol` 0.01 mm, accuracy ± 2·0.01 mm); if the parts intersect, the distance is 0 and `collision: true`.

Expected values, spec example *gehaeuse*: k.ecke = [60, 30, 20], k.oben = [0, 0, 20] → √(60² + 30²) = 67.082 mm; in the module g[links,vorne,boden] → g[rechts,hinten,deckel] = √(40² + 30² + 20²) = 53.852 mm; volume k = 60·30·20 − 54·30·17 = 8460 mm³. *kollision.scad*: a and b overlap by 2 × 10 × 10 = 200 mm³, c is 5 mm next to b.

$ cad measure crates/cad-cli/tests/data/gehaeuse.scad --between k.ecke k.oben --bounds part:k --volume part:k --area part:k 40 ms, Exit 0

```
Abstand k.ecke [60, 30, 20] → k.oben [0, 0, 20]: 67.082 mm (Δ [-60, -30, 0])
Hülle part:k: [0, 0, 0] – [60, 30, 20], Größe 60 × 30 × 20 mm
Volumen part:k: 8460 mm³
Fläche part:k: 6384 mm²
```

$ cad measure crates/cad-cli/tests/data/gehaeuse.scad --module gehaeuse --between "g[links,vorne,boden]" "g[rechts,hinten,deckel]" 13 ms, Exit 0

```
Abstand g[links,vorne,boden] [0, 0, 0] → g[rechts,hinten,deckel] [40, 30, 20]: 53.8516 mm (Δ [40, 30, 20])
```

$ cad measure crates/cad-cli/tests/data/gehaeuse.scad --module gehaeuse -D w=60 --between ecke "[0,0,0]" 13 ms, Exit 0

```
Abstand ecke [60, 30, 20] → [0,0,0] [0, 0, 0]: 70 mm (Δ [-60, -30, -20])
```

$ cad measure crates/cad-cli/tests/data/gehaeuse.scad --between "g[links,vorne,boden]" "[0,0,0]" 37 ms, Exit 2

Error expected: grid g is not visible in file context (exit 2).

```
cad measure: g[links,vorne,boden]: Raster `g` ist nicht im Kontext (Raster gelten nur in ihrem Modul: --module NAME)
```

$ cad measure crates/cad-cli/tests/data/kollision.scad --interference --clearance b c 22 ms, Exit 0

```
Kollision a ↔ b: 200 mm³
Abstand b ↔ c: 5 mm (± 0.02 mm, Tessellierung 0.01 mm), zwischen [18, 10, 0] und [23, 10, 0]
```

$ cad measure crates/cad-cli/tests/data/kollision.scad --interference --json 33 ms, Exit 0

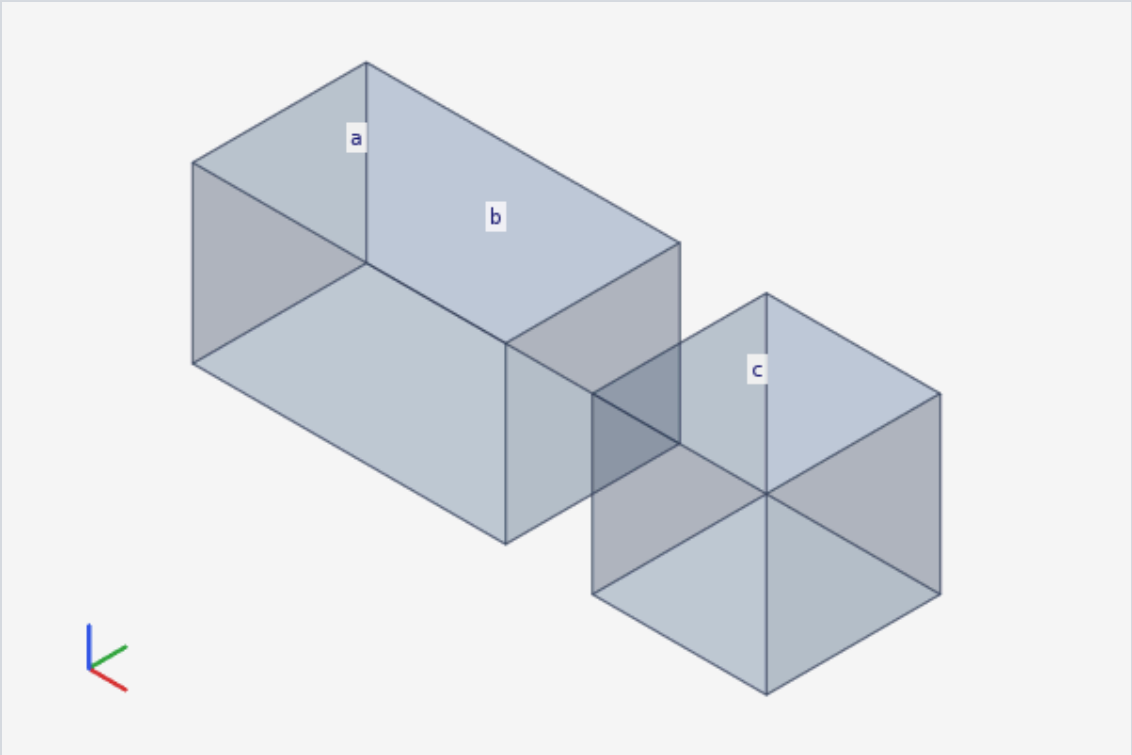
JSON (EXCERPT: INTERFERENCE)


```
{
  "interference": {
    "collisions": [{"a": "a", "b": "b", "volume": 200}],
    "method": "kernel intersection (exact B-rep)",
    "pairs": [
      {"a": "a", "b": "b", "bounds_overlap": true, "collision": true, "volume": 200},
      {"a": "a", "b": "c", "bounds_overlap": false, "volume": 0},
      {"a": "b", "b": "c", "bounds_overlap": false, "volume": 0}
    ],
    "parts": ["a", "b", "c"],
    "tolerance": 0.001
  }
}
```


$ cad measure examples/projekt-gehaeuse/gehaeuse.scad --interference --clearance gehaeuse deckel --json 488 ms, Exit 0

JSON (EXCERPT: CLEARANCE)


```
{
  "clearance": {
    "a": "gehaeuse",
    "accuracy": 0.02,
    "b": "deckel",
    "collision": false,
    "distance": 10,
    "intersection_volume": 0,
    "mesh_distance": 10,
    "mesh_tol": 0.01,
    "method": "minimum triangle–triangle distance of both tessellations",
    "point_a": [60, 40, 0],
    "point_b": [60, 50, 0],
    "triangles": [1820, 28]
  }
}
```



kollision.scad --xray --show labels: a and b overlap, c at 5 mm distance

7 cad set

`--param modul.param=wert` writes a default value (like "Use as default" (German UI: „Als Standard übernehmen“) in the UI, also for modules from `use` files), `--literal [datei:]zeile:spalte=wert` replaces the editable literal at that position – the same literals the UI changes via handles, sliders and grid planes. Before writing, the changed project is evaluated; new errors prevent writing (exit 1, `--force` overrides). Writing is atomic (temporary file + rename); `--dry-run` only shows the diff.

`$ cad set reports/tmp/report8/work/set.scad --param gehaeuse.w=50 --literal 21:17=70 --dry-run` 6 ms, Exit 0

```
--- reports/tmp/report8/work/set.scad
+++ reports/tmp/report8/work/set.scad
@@ -1,5 +1,5 @@
 // Zweck: Spec-Beispiel „gehaeuse“ – Raster g im Modul, Box zwischen Rasterpunkten, Anker
 // nach außen, benannte Instanz k, Deckel am Anker und ein zweites Raster aus k.ecke.x.
-module gehaeuse(w = 40, d = 30, h = 20, t = 3) {
+module gehaeuse(w = 50, d = 30, h = 20, t = 3) {
     grid g {
         x: links = 0, innen_l = t, innen_r = w - t, rechts = w;
@@ -19,5 +19,5 @@
 module deckel() cube([10, 10, 2]);

-k: gehaeuse(w = 60);
+k: gehaeuse(w = 70);
 translate(k.ecke) deckel();
 translate(k.oben) rotate(to = dir(k.oben)) deckel();
(--dry-run: nichts geschrieben)
```

`$ cad set reports/tmp/report8/work/set.scad --param gehaeuse.w=50 --literal 21:17=70 --json` 7 ms, Exit 0

JSON (EXCERPT: CHANGES, WRITTEN, ERRORS_BEFORE, ERRORS_AFTER)

```
{
  "changes": [
    {
      "col": 21,
      "file": "reports/tmp/report8/work/set.scad",
      "kind": "param",
```

```
      "line": 3,
      "new": "50",
      "old": "40",
      "spec": "gehaeuse.w=50"
    },
    {
      "col": 17,
      "file": "reports/tmp/report8/work/set.scad",
      "kind": "literal",
      "line": 21,
      "new": "70",
      "old": "60",
      "spec": "21:17=70"
    }
  ],
  "written": ["reports/tmp/report8/work/set.scad"],
  "errors_before": 0,
  "errors_after": 0
}
```


$ cad set reports/tmp/report8/work/set.scad --literal 5:30=4 6 ms, Exit 2

Error expected: 5:30 holds `t`, a reference to a parameter, not a literal (exit 2).

```
cad set: --literal 5:30=4: an 5:30 steht kein editierbares Literal (Literale der Zeile: 5:20 `0`)
```

$ cad set reports/tmp/report8/work/set.scad --param gehaeuse.t=foo 8 ms, Exit 1

Rejected: the change produces errors (exit 1), the file stays unchanged.

```
--- reports/tmp/report8/work/set.scad
+++ reports/tmp/report8/work/set.scad
@@ -1,5 +1,5 @@
 // Zweck: Spec-Beispiel „gehaeuse“ – Raster g im Modul, Box zwischen Rasterpunkten, Anker
 // nach außen, benannte Instanz k, Deckel am Anker und ein zweites Raster aus k.ecke.x.
-module gehaeuse(w = 50, d = 30, h = 20, t = 3) {
+module gehaeuse(w = 50, d = 30, h = 20, t = foo) {
     grid g {
         x: links = 0, innen_l = t, innen_r = w - t, rechts = w;
nicht geschrieben: die Änderung erzeugt Fehler (0 → 5); --force schreibt trotzdem
reports/tmp/report8/work/set.scad:3:45: Warnung (Auswertung): unbekannte Variable `foo`
reports/tmp/report8/work/set.scad:5:33: Fehler (Auswertung): Ebene `innen_l` muss Zahl oder Zahlenliste sein,
ist undef
reports/tmp/report8/work/set.scad:5:46: Warnung (Auswertung): undefinierte Operation (Zahl - undef)
reports/tmp/report8/work/set.scad:5:46: Fehler (Auswertung): Ebene `innen_r` muss Zahl oder Zahlenliste sein,
ist undef
reports/tmp/report8/work/set.scad:12:15: Fehler (Auswertung): Raster `g` hat keine Ebene `innen_l`
reports/tmp/report8/work/set.scad:12:37: Fehler (Auswertung): Raster `g` hat keine Ebene `innen_r`
reports/tmp/report8/work/set.scad:12:13: Warnung (Auswertung): `a` muss ein Vektor [x, y, z] sein, ist undef
reports/tmp/report8/work/set.scad:12:35: Warnung (Auswertung): `b` muss ein Vektor [x, y, z] sein, ist undef
reports/tmp/report8/work/set.scad:12:9: Fehler (Auswertung): box() braucht zwei Eckpunkte `a` und `b`
```

8 Reserved UI entry point

`cad datei.scad` (a positional argument that is not a command) will later start the desktop UI. Today: message and exit 2. `cad` without arguments shows the help (exit 0). A typo in the command gives "unknown command" („unbekannter Befehl“).

$ cad crates/cad-cli/tests/data/gehaeuse.scad 5 ms, Exit 2

```
cad: UI nicht in diesem Build (`cad crates/cad-cli/tests/data/gehaeuse.scad` startet später die Desktop-UI).
Befehle: check, outline, eval, inspect, render, measure, watch, set, export, diff-kernels
Beispiel: cad render crates/cad-cli/tests/data/gehaeuse.scad -o bild.png --views all
```

$ cad 5 ms, Exit 0

```
Parametrisches CAD: prüfen, auswerten, ansehen, messen, exportieren

Usage: cad [COMMAND]

Commands:
  check         Diagnosen mit Zeile und Spalte
  outline       Übersicht in einem Aufruf: Module mit Parametern, Funktionen, Variablen, Raster und Anker je
Modul, benannte Instanzen, Bauteile, Dateien, Diagnosen
  eval          Baum, Werte, Inline-Hinweise, Raster, Anker
  inspect       Was eine Zeile erzeugt: Knoten, Hülle, Volumen, ausgewählte Kanten, Meldungen
  render        Bild des Modells (PNG): Ansichten, Kontaktbogen, freie Kamera, Auswahl, Überlagerungen,
Schnitt, Röntgen
  measure       Messen: Abstand zweier Punkte (Anker, Rasterpunkt, Koordinate), Hülle, Volumen, Fläche,
Kollisionen der Bauteile, Abstand zweier Bauteile
  watch         Beobachtet die Datei und eingebundene Dateien; je Auswertung eine JSON-Zeile (mit --json) bzw.
eine Textzeile, optional neues Bild
  set           Code ändern über die Engine: Standardwert eines Modulparameters oder ein Literal; schreibt
atomar (`--dry-run`: nur Diff)
  export        Exportdatei (STL binär, 3MF oder STEP, nach Endung oder --format)
  diff-kernels  Vergleich SMLib ↔ solidcore je Knoten (Volumen, Flächen, Kanten, ausgewählte Kanten) und
Abdeckung von solidcore (nur Entwicklung)
  help          Print this message or the help of the given subcommand(s)

Options:
  -h, --help     Print help
  -V, --version  Print version

Ablauf für Agenten: outline → editieren → check → render --views all --show grids,anchors → inspect/measure →
export. `cad watch` wertet bei jeder Dateiänderung neu aus (Cache bleibt).
… (1 Zeilen mehr)
```

9 cad watch

`cad watch DATEI [--json] [--render bild.png …]` watches the file and all included files (crate `notify`, inotify on Linux; the directories are watched so that atomic saves by editors are detected too), re-evaluates in the same session on every change and writes one JSON line per run (diagnostics, timings, volume, parts, image). The engine cache is kept: unchanged nodes come from the cache. Stop with Ctrl+C, `--once-after-change` or `--timeout S`.

File	Run	Trigger	Total ms	Kernel ms	Image ms	from cache / nodes	Kernel calls	Volume mm³
variabler_radius.scad	1: start (everything new)	start	1,101.9	734.3	367.1	1 / 12	29	56,527.5
variabler_radius.scad	2: comment appended (everything from the cache)	variabler_radius.scad	29.7	0.0	28.9	12 / 12	0	56,527.5
variabler_radius.scad	3: deckel = 22 (two fillets rebuilt)	variabler_radius.scad	947.1	569.0	377.5	6 / 12	16	61,297.6
variabler_radius.scad	4: radius function of the pocket changed	variabler_radius.scad	329.6	103.0	226.0	9 / 12	9	56,573.0
projekt-gehaeuse	1: start (everything new)	start	252.5	164.7	–	9 / 33	29	26,212.8
projekt-gehaeuse	2: konstanten.scad: abstand = 15 (only position of the lid)	konstanten.scad	6.5	5.3	–	31 / 33	2	26,212.8

projekt-gehaeuse	3: lib/teile.scad: dom_r = 3.5 (screw bosses rebuilt)	lib/teile.scad	210.0	126.3	–	20 / 33	11	26,988.8

JSON LINE (RUN 3, SHORTENED)


```
{
  "command": "watch",
  "run": 3,
  "trigger": "change",
  "changed": ["variabler_radius.scad"],
  "ok": true,
  "errors": 0,
  "stats": {"cache_hits": 6, "kernel_calls": 16, "nodes": 12},
  "volume": 61297.5811,
  "timings": {"eval_ms": 0.2223, "kernel_ms": 569.007, "render_ms": 377.4542, "total_ms": 947.1355},
  "render": {"height": 400, "output": "docs/reports/08-cli-agenten/img/watch.png", "width": 800}
}
```

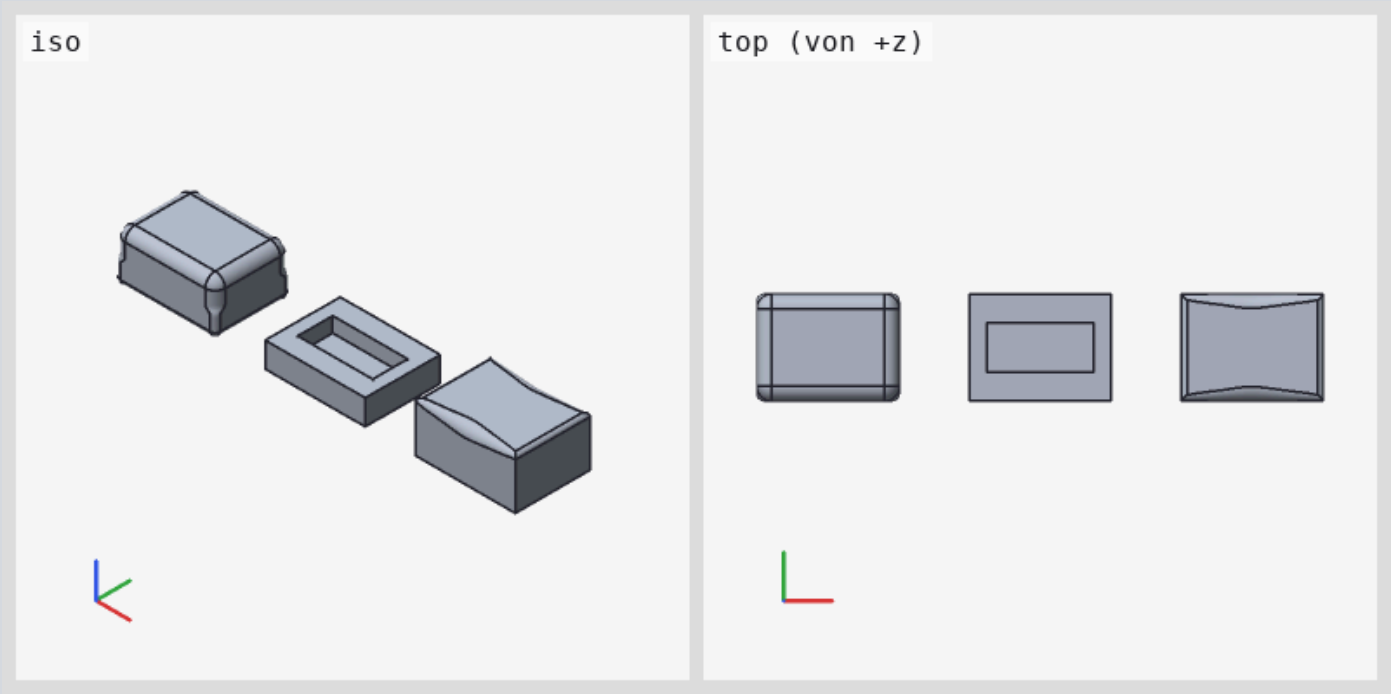



Image after the last run of watch --render … --views iso,top

10 Performance of repeated calls

Every CLI call starts a new process with an empty cache. Measured: `cad render` twice in a row (default view 1024×768), wall time of the process and the timings from the JSON (`timings`: evaluation incl. kernel, of which kernel, image incl. tessellation). Process start with parser, without kernel (`check --no-kernel`): 7 ms.

File	Run	Wall time ms	Evaluation ms	of which kernel ms	Image ms
spec/variabler_radius.scad	1	1,017	631	630	375
spec/variabler_radius.scad	2	1,104	667	666	426
projekt-gehaeuse/gehaeuse.scad	1	579	340	218	229
projekt-gehaeuse/gehaeuse.scad	2	610	378	248	220
spec/gehaeuse.scad	1	159	37	34	114
spec/gehaeuse.scad	2	165	30	29	126

Decision: no persistent cache between calls. The second call is as fast as the first (no cache); a call takes up to 1.10 s – for *variabler_radius* at the 1 s limit, almost all of it kernel time (fillets) and tessellation. A file cache of the

kernel results would be costly: the results are handles to SMLib objects in the memory of the native library; a cache would have to serialize and re-read the B-rep (together with the provenance of faces and edges), which itself costs kernel time. Instead `cad watch`: the same session persists, the engine cache holds all node results; a run after a change costs only the changed nodes (table in section 9: 30 ms for a change without effect on geometry, 947 ms when two filleted bodies are rebuilt). The skill recommends `cad watch` for loops with many small steps.

Run time per command (spec example gehaeuse, wall time)

Command	ms	Exit
outline	46	0
check	44	0
render (iso)	143	0
render --views all --show grids,anchors	170	0
render --section z=10	285	0
measure --between --volume	35	0
measure --interference (kollision)	25	0
measure --clearance b c (kollision)	19	0
set --dry-run	6	0

11 Case study: an agent develops a part using only the CLI

The agent (this report was produced by Claude) designed a wall bracket for a Ø 20 mm rod without opening the UI: source code with the Edit tool, everything else with `cad`. The steps below are the ones actually executed, replayed by the generator (versions in `tools/report/report8-fallstudie/`). The agent viewed the images with the Read tool; a real design error was found (no clearance between rod and bore), visible only through `measure --clearance`, not in the image.

Step 1: write the draft

First version with grid g (planes for plate, arm, bore, screw holes), two anchors and the parts halter and stange.

SOURCE CODE

```
// Fallstudie Bericht 8: Wandhalter für eine Stange, nur mit der CLI entwickelt.
// Grundplatte an der Wand (y = 0), Ausleger mit Bohrung für die Stange, zwei Schraublöcher.
module halter(b = 40, t = 5, h = 50, a = 30, d = 20, spiel = 0) {
    grid g {
        x: links = 0, loch_l = 10, mitte = b / 2, loch_r = b - 10, rechts = b;
        y: wand = 0, platte = t, achse = t + a, vorn = t + a + d / 2 + t;
        z: unten = 0, loch = 10, arm = h - d - 2 * t, achse_z = h - t - d / 2, oben = h;
    }
    difference() {
        union() {
            box(g[links, wand, unten], g[rechts, platte, oben]);
            box(g[links, platte, arm], g[rechts, vorn, oben]);
        }
        cylinder(g[-1, achse, achse_z], g[b + 1, achse, achse_z], r = (d + spiel) / 2);
        for (x = [g.loch_l, g.loch_r])
            cylinder(g[x, -1, loch], g[x, t + 1, loch], r = 2.25);
    }
    anchor stange = g[links, achse, achse_z], dir = [1, 0, 0];
    anchor schraube = g[loch_l, wand, loch], dir = [0, -1, 0];
}

module stange(l = 60, d = 20) cylinder(h = l, r = d / 2);
```

```
halter: halter();
translate(halter.stange) rotate(to = dir(halter.stange)) translate([0, 0, -10]) stange: stange();
```

Step 2: check and understand

check reports no errors; outline shows grid, anchors and parts with bounding box and volume.

$ cad check reports/tmp/report8/work/wandhalter.scad 159 ms, Exit 0

```
reports/tmp/report8/work/wandhalter.scad: 0 Fehler, 0 Warnungen
```

$ cad outline reports/tmp/report8/work/wandhalter.scad 146 ms, Exit 0

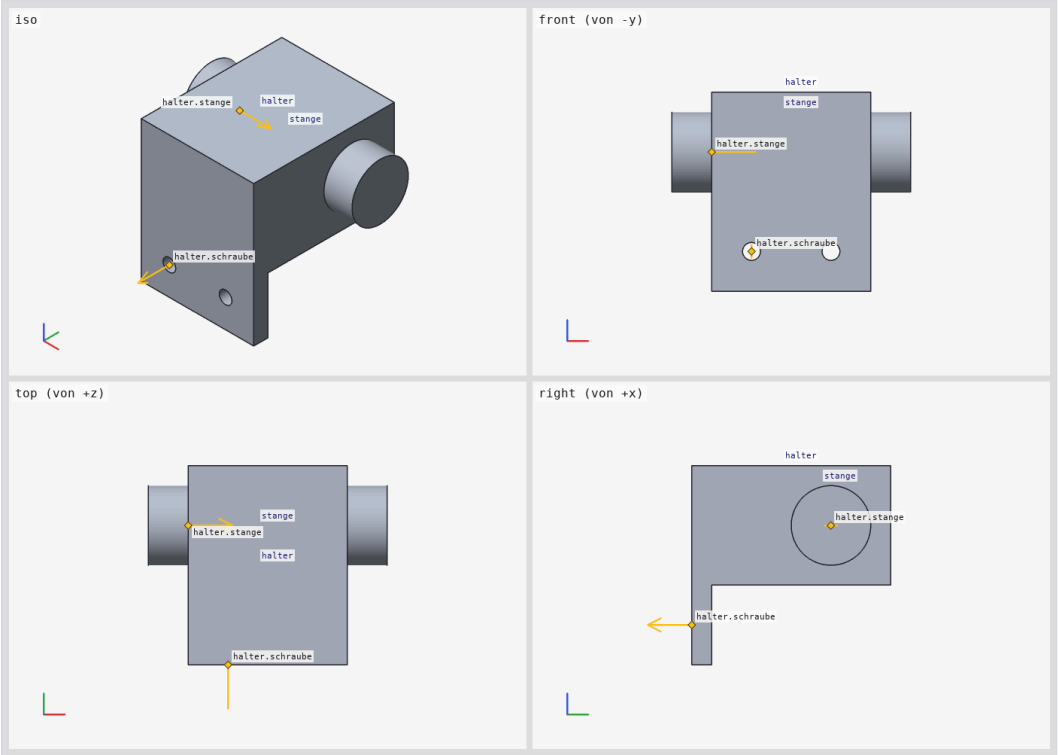
```
reports/tmp/report8/work/wandhalter.scad: 0 Fehler, 0 Warnungen
Module:
  halter(b = 40: Zahl, t = 5: Zahl, h = 50: Zahl, a = 30: Zahl, d = 20: Zahl, spiel = 0: Zahl)
wandhalter.scad:3–20
    Raster g (Z.4): x: links = 0, loch_l = 10, mitte = b / 2 → 20, loch_r = b - 10 → 30, rechts = 40; y:
wand = 0, platte = 5, achse = t + a → 35, vorn = t + a + d / 2 + t → 50; z: unten = 0, loch = 10, arm = h -
d - 2 * t → 20, achse_z = h - t - d / 2 → 35, oben = 50
    Anker: stange = [0, 35, 35] dir [1, 0, 0] (Z.18), schraube = [10, 0, 10] dir [0, -1, 0] (Z.19)
  stange(l = 60: Zahl, d = 20: Zahl)  wandhalter.scad:22–22
Benannte Instanzen:
  halter: halter  Z.24  Bauteil, Hülle [0, 0, 0]–[40, 50, 50], Volumen 51274.5869 mm³
  stange: stange  Z.25  Bauteil, Hülle [-10, 25, 25]–[50, 45, 45], Volumen 18849.5559 mm³
Anker: halter.stange = [0, 35, 35] dir [1, 0, 0], halter.schraube = [10, 0, 10] dir [0, -1, 0]
Gesamt: Hülle [-10, 0, 0]–[50, 50, 50] (60 × 50 × 50 mm), 15 Knoten
```

Step 3: look

Contact sheet with anchors and labels: plate, arm, bore and rod are positioned as planned.

$ cad render reports/tmp/report8/work/wandhalter.scad -o docs/reports/08-cli-agenten/img/fs-1.png --views all --show "anchors,labels" --size 1400x1000 339 ms, Exit 0

```
docs/reports/08-cli-agenten/img/fs-1.png: 1400x1000, Ansichten iso, front, top, right, Hülle 60 × 50 × 50
mm
```



Step 4: measure

Collisions and clearance rod ↔ bore: distance 0 – the rod sits in the bore without clearance (Ø 20 in Ø 20). Design error found.

```
$ cad measure reports/tmp/report8/work/wandhalter.scad --interference --clearance halter stange
```
318 ms, Exit 0

```
Keine Kollisionen (2 Bauteile, 1 Paare mit überlappender Hülle geprüft, Toleranz 0.001 mm³)
Abstand halter ↔ stange: 0 mm (± 0.02 mm, Tessellierung 0.01 mm), zwischen [40, 31.7825, 44.4683] und [40, 31.7825, 44.4683]
```

```
$ cad measure reports/tmp/report8/work/wandhalter.scad --between halter.schraube halter.stange
```
245 ms, Exit 0

```
Abstand halter.schraube [10, 0, 10] → halter.stange [0, 35, 35]: 44.1588 mm (Δ [-10, 35, 25])
```

Step 5: fix with cad set

Set a clearance of 0.5 mm as default value (diff first, then write); afterwards the distance is 0.25 mm (radial) ± 0.02 mm.

```
$ cad set reports/tmp/report8/work/wandhalter.scad --param halter.spiel=0.5 --dry-run
```
7 ms, Exit 0

```
--- reports/tmp/report8/work/wandhalter.scad
+++ reports/tmp/report8/work/wandhalter.scad
@@ -1,5 +1,5 @@
 // Fallstudie Bericht 8: Wandhalter für eine Stange, nur mit der CLI entwickelt.
 // Grundplatte an der Wand (y = 0), Ausleger mit Bohrung für die Stange, zwei Schraublöcher.
-module halter(b = 40, t = 5, h = 50, a = 30, d = 20, spiel = 0) {
+module halter(b = 40, t = 5, h = 50, a = 30, d = 20, spiel = 0.5) {
     grid g {
         x: links = 0, loch_l = 10, mitte = b / 2, loch_r = b - 10, rechts = b;
(--dry-run: nichts geschrieben)
```

```
$ cad set reports/tmp/report8/work/wandhalter.scad --param halter.spiel=0.5
```
7 ms, Exit 0

```
--- reports/tmp/report8/work/wandhalter.scad
+++ reports/tmp/report8/work/wandhalter.scad
@@ -1,5 +1,5 @@
 // Fallstudie Bericht 8: Wandhalter für eine Stange, nur mit der CLI entwickelt.
 // Grundplatte an der Wand (y = 0), Ausleger mit Bohrung für die Stange, zwei Schraublöcher.
-module halter(b = 40, t = 5, h = 50, a = 30, d = 20, spiel = 0) {
+module halter(b = 40, t = 5, h = 50, a = 30, d = 20, spiel = 0.5) {
     grid g {
         x: links = 0, loch_l = 10, mitte = b / 2, loch_r = b - 10, rechts = b;
geschrieben: reports/tmp/report8/work/wandhalter.scad
```

$ cad measure reports/tmp/report8/work/wandhalter.scad --clearance halter stange 368 ms, Exit 0

```
Abstand halter ↔ stange: 0.2499 mm (± 0.02 mm, Tessellierung 0.01 mm), zwischen [40, 41.8678, 27.3913] und
[40, 41.6957, 27.5725]
```

Step 6: fillet the inner edge

Zone block fillet(4) around the inner edge plate/arm (zone(g[*, platte, arm], d = 2)), inserted with the Edit tool. inspect of the fillet line: exactly 1 edge selected.

CHANGE (DIFF)

```
--- vorher
+++ nachher
@@ -7,9 +7,12 @@
         z: unten = 0, loch = 10, arm = h - d - 2 * t, achse_z = h - t - d / 2, oben = h;
     }
     difference() {
-        union() {
-            box(g[links, wand, unten], g[rechts, platte, oben]);
-            box(g[links, platte, arm], g[rechts, vorn, oben]);
+        fillet(4) {
+            union() {
+                box(g[links, wand, unten], g[rechts, platte, oben]);
+                box(g[links, platte, arm], g[rechts, vorn, oben]);
+            }
+            zone(g[*, platte, arm], d = 2);         // Innenkante Platte/Ausleger
         }
         cylinder(g[-1, achse, achse_z], g[b + 1, achse, achse_z], r = (d + spiel) / 2);
         for (x = [g.loch_l, g.loch_r])
```

$ cad check reports/tmp/report8/work/wandhalter.scad 273 ms, Exit 0

```
reports/tmp/report8/work/wandhalter.scad: 0 Fehler, 0 Warnungen
```

$ cad inspect reports/tmp/report8/work/wandhalter.scad --line 10 119 ms, Exit 0

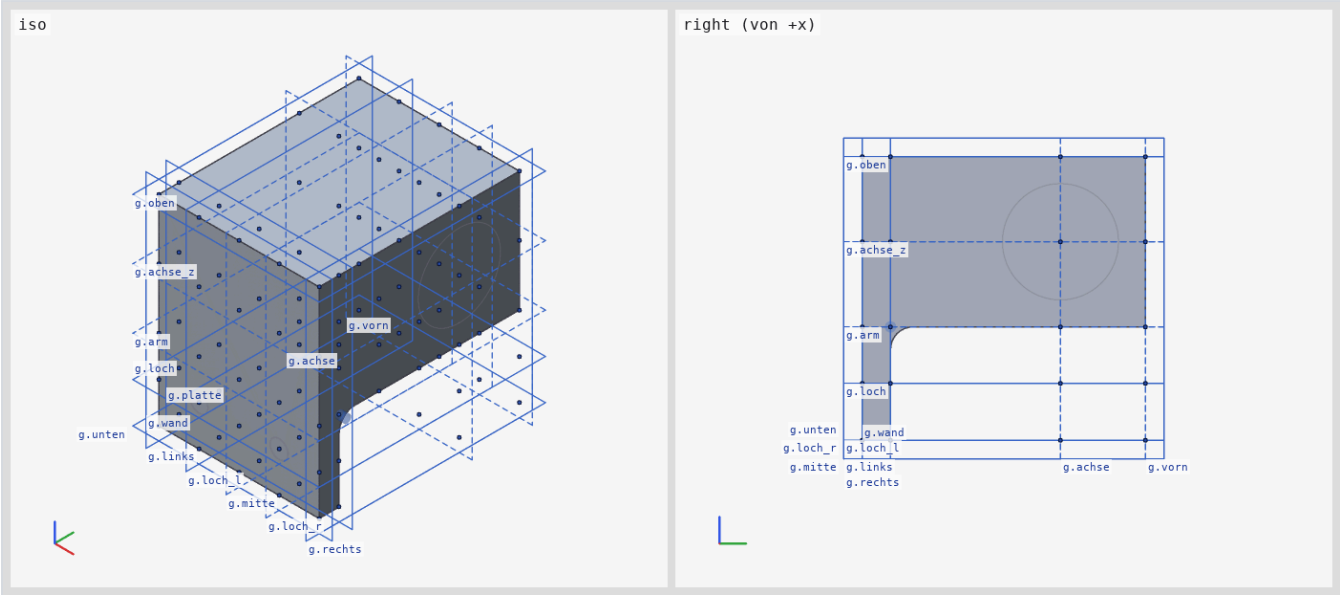
```
Kontext: Modul halter(b = 40, t = 5, h = 50, a = 30, d = 20, spiel = 0.5)
Z.10: fillet(4) {
  [2] fillet  Z.10:9–16:10  ok
      Hülle [0, 0, 0] – [40, 50, 50]
      Volumen 64137.3452 mm³, Fläche 11138.1947 mm²
      ausgewählte Kanten: 1
      Hinweis: 1 Zone
      Hinweis: 1 Kante
```

Step 7: view grid and zone

Line 10 highlighted in module context, grid with plane names (100 grid points: only planes labelled).

$ cad render reports/tmp/report8/work/wandhalter.scad -o docs/reports/08-cli-agenten/img/fs-2.png --line 10 --views "iso,right" --show "grids,labels" --size 1400x620 652 ms, Exit 0

```
docs/reports/08-cli-agenten/img/fs-2.png: 1400x620, Ansichten iso, right, Zeile 10: [2] fillet, Kontext
Modul halter, Hülle 40 × 50 × 50 mm
Hinweis: Raster g: 100 Punkte, nur Ebenen beschriftet (Punkte bis 48; --view und --zoom für Details)
```

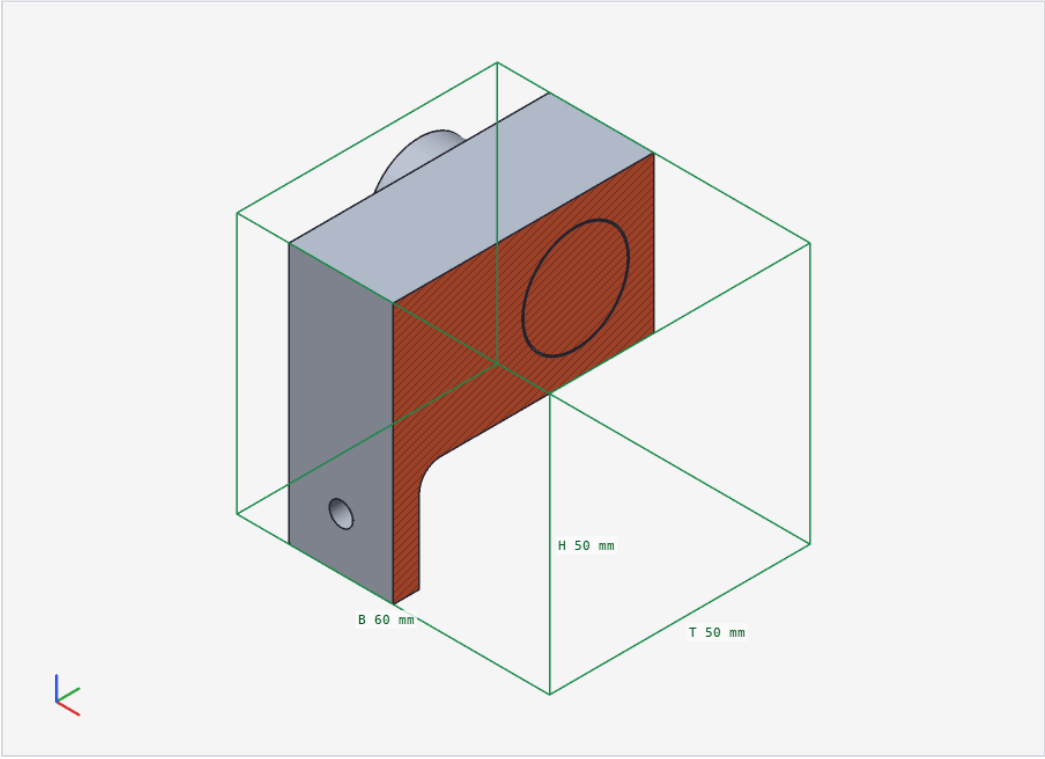


Step 8: section through the bore

Section x = 20 shows rod and bore with gap; dimensions of the whole.

```
$ cad render reports/tmp/report8/work/wandhalter.scad -o docs/reports/08-cli-agenten/img/fs-3.png --section
x=20 --show dims --size 900x650                                                          866 ms, Exit 0
```

```
docs/reports/08-cli-agenten/img/fs-3.png: 900x650, Ansicht iso, Schnitt durch [20, 0, 0] Normale [1, 0, 0],
Hülle 60 × 50 × 50 mm
```



Step 9: export

Each part as a STEP file.

```
$ cad export reports/tmp/report8/work/wandhalter.scad --all-parts -o reports/tmp/report8/work/teile --
format step                                                                          185 ms, Exit 0
```

```
halter: reports/tmp/report8/work/teile/halter.step (0 Dreiecke, 14823 Bytes, Volumen 50775.7597 mm³)
stange: reports/tmp/report8/work/teile/stange.step (0 Dreiecke, 3821 Bytes, Volumen 18849.5559 mm³)
```

Result: 13 CLI calls, 3.7 s in total. What would have been missing without the new features: the clearance measurement, the grid names in the image (which plane is `arm`?) and the section through the bore.

12 Tests

- `cargo test --workspace`: 318 passed, 0 failed, 0 ignored; `cargo clippy --workspace --all-targets -- -D warnings`: no findings; `cargo fmt --check`: no findings.
- New in `crates/cad-cli/tests/agent.rs` (assert_cmd, per command/option with JSON fields): outline (spec example, functions/variables, project), render (contact sheet: image size and equal scale; free camera; selection; overlays change pixels; section reduces the visible area and produces cap pixels; X-ray), measure (anchors, grid points, coordinates against expected values of the spec example; bounding box, volume; collision on `tests/data/kollision.scad`; clearance), watch (change file → second JSON line, cache hits increase; text and timeout), set (parameter, literal, diff, rejection, project files), UI entry point.
- New in `crates/cad-render/tests/features.rs`: camera angles = standard views, section with cap on a closed cube (all cap from above, none from below), perspective and fixed scale, markers/labels/title/contact sheet; in `geom.rs`: triangle–triangle distance on cubes (face, edge, vertex, overlap).

Test file	Test	Result
unittests src/main.rs	geom::tests::distance_of_cubes	ok
tests/agent.rs	outline_spec_example	ok
tests/agent.rs	measure_points_bounds_volume	ok
tests/agent.rs	outline_functions_variables_and_projects	ok
tests/agent.rs	measure_interference_and_clearance	ok
tests/agent.rs	render_free_camera	ok
tests/agent.rs	render_section_reduces_visible_area	ok
tests/agent.rs	render_overlays_change_pixels	ok
tests/agent.rs	render_contact_sheet	ok
tests/agent.rs	set_param_and_literal	ok
tests/agent.rs	ui_entry_is_reserved	ok
tests/agent.rs	render_xray	ok
tests/agent.rs	set_in_project_file	ok
tests/agent.rs	watch_text_and_timeout	ok
tests/agent.rs	watch_reports_a_second_run_after_a_change	ok
tests/agent.rs	render_selection	ok
tests/features.rs	angles_match_named_views	ok
tests/features.rs	perspective_and_fixed_scale	ok
tests/features.rs	markers_labels_title_and_sheet	ok

tests/features.rs	section_clips_and_caps_a_closed_mesh	ok

13 Limitations

- Section caps require closed meshes; open surfaces (`surface` without `thicken`) and transparent bodies get no cap. The cap is flat and single-colored (hatching alternates per body), no distinction by material.
- `--clearance` measures on the tessellation (accuracy ± 2·`--mesh-tol`), `--interference` exactly via the kernel; parts whose bounding boxes do not overlap are counted as collision-free without the kernel.
- Grid points are labelled up to 48 points per grid, above that only the planes (note in `notes`); overlapped labels are dropped.
- `outline` evaluates each module with its default values; modules with required parameters without a default may then show no grids (`errors_with_defaults`).
- `watch` re-reads all project files on every change; changes within the debounce time (`--debounce`, 60 ms) are merged into one run.
- No persistent cache between calls (section 10).